

Lecture 9

Interface and Implementation Inheritance

CS61B, Spring 2026 @ UC Berkeley

Josh Hug and Kay Ousterhout

The Desire for Generality

Lecture 9, CS61B, Spring 2026

Interface Inheritance

- **The Desire for Generality**
- Hypernyms and Hyponyms
- Interface and Implements
- Keywords
- Overriding vs. Overloading
- Interface Inheritance

Implementation Inheritance

- Default Methods
- Overriding Default Methods
- vs. Interface Inheritance

Collection Abstract Data Types

Changes to Scope in 61B

After adding an additional “insert” method. Our AList and SLList classes from lecture have the following methods (exact same method signatures for both classes).

```
public class AList<Item>{  
    public AList()  
    public void insert(Item x, int position)  
    public void addFirst(Item x)  
    public void addLast(Item i)  
    public Item getFirst()  
    public Item getLast()  
    public Item get(int i)  
    public int size()  
    public Item removeLast()  
}
```

```
public class SLList<Blorp>{  
    public SLList()  
    public SLList(Blorp x)  
    public void insert(Blorp item, int position)  
    public void addFirst(Blorp x)  
    public void addLast(Blorp x)  
    public Blorp getFirst()  
    public Blorp getLast()  
    public Blorp get(int i)  
    public int size()  
    public Blorp removeLast()  
}
```

Suppose we're writing a library to manipulate lists of words. Might want to write a function that finds the longest word from a list of words:

```
public static String longest(SLList<String> list) {  
    int maxDex = 0;  
    for (int i = 0; i < list.size(); i += 1) {  
        String longestString = list.get(maxDex);  
        String thisString = list.get(i);  
        if (thisString.length() > longestString.length()) {  
            maxDex = i;  
        }  
    }  
  
    return list.get(maxDex);  
}
```

Observant viewers may note this code is very inefficient! Don't worry about it.

If we want longest to be able to handle ALists, what changes do we need to make?

```
public static String longest(SLList<String> list) {  
    int maxDex = 0;  
    for (int i = 0; i < list.size(); i += 1) {  
        String longestString = list.get(maxDex);  
        String thisString = list.get(i);  
        if (thisString.length() > longestString.length()) {  
            maxDex = i;  
        }  
    }  
  
    return list.get(maxDex);  
}
```

If we want longest to be able to handle ALists, what changes do we need to make?



```
public static String longest(ALList<String> list) {  
    int maxDex = 0;  
    for (int i = 0; i < list.size(); i += 1) {  
        String longestString = list.get(maxDex);  
        String thisString = list.get(i);  
        if (thisString.length() > longestString.length()) {  
            maxDex = i;  
        }  
    }  
  
    return list.get(maxDex);  
}
```

Method Overloading in Java

Java allows multiple methods with same name, but different parameters.

- This is called method **overloading**.

```
public static String longest(AList<String> list) {  
    ...  
}  
  
public static String longest(SLList<String> list) {  
    ...  
}
```

While overloading works, it is a bad idea in the case of `longest`. Why?

- Code is virtually identical. Aesthetically gross.
- Won't work for future lists. If we create a `QList` class, have to make a third method.
- Harder to **maintain**.
 - Example: Suppose you find a bug in one of the methods. You fix it in the `SLList` version, and forget to do it in the `AList` version.

Hypernyms and Hyponyms

Lecture 9, CS61B, Spring 2026

Interface Inheritance

- The Desire for Generality
- **Hypernyms and Hyponyms**
- Interface and Implements Keywords
- Overriding vs. Overloading
- Interface Inheritance

Implementation Inheritance

- Default Methods
- Overriding Default Methods
- vs. Interface Inheritance

Collection Abstract Data Types

Changes to Scope in 61B

In natural languages (English, Spanish, Chinese, Tagalog, etc.), we have a concept known as a “hypernym” to deal with this problem.

- Dog is a “hypernym” of poodle, malamute, yorkie, etc.

Washing your poodle:

1. Brush your poodle before a bath. ...
2. Use lukewarm water. ...
3. Talk to your poodle in a calm voice. ...
4. Use poodle shampoo. ...
5. Rinse well. ...
6. Air-dry. ...
7. Reward your poodle.

Washing your **dog**:

1. Brush your **dog** before a bath. ...
2. Use lukewarm water. ...
3. Talk to your **dog** in a calm voice. ...
4. Use dog shampoo. ...
5. Rinse well. ...
6. Air-dry. ...
7. Reward your **dog**.

Washing your malamute:

1. Brush your malamute before a bath. ...
2. Use lukewarm water. ...
3. Talk to your malamute in a calm voice. ...
4. Use malamute shampoo. ...
5. Rinse well. ...
6. Air-dry. ...
7. Reward your malamute.

Hypernym and Hyponym

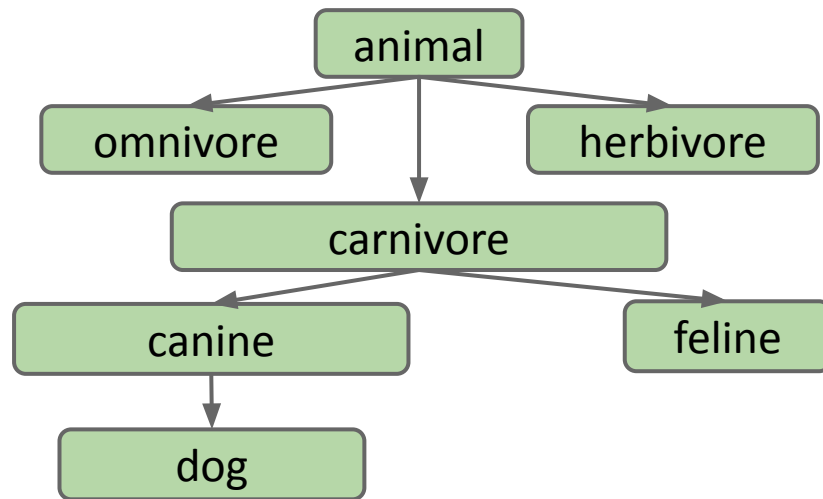
We use the word hyponym for the opposite type of relationship.

- “dog”: Hyponym of “poodle”, “malamute”, “dachshund”, etc.
- “poodle”: Hyponym of “dog”

Hypernyms and hyponyms comprise a hierarchy.

- A dog “is-a” canine.
- A canine “is-a” carnivore.
- A carnivore “is-an” animal.

(for fun: see the [WordNet project](#))



Interface and Implements Keywords

Lecture 9, CS61B, Spring 2026

Interface Inheritance

- The Desire for Generality
- Hypernyms and Hyponyms
- **Interface and Implements Keywords**
- Overriding vs. Overloading
- Interface Inheritance

Implementation Inheritance

- Default Methods
- Overriding Default Methods
- vs. Interface Inheritance

Collection Abstract Data Types

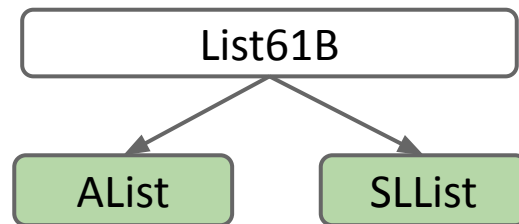
Changes to Scope in 61B

SLLists and ALists are both clearly some kind of “list”.

- List is a hypernym of SLList and AList.

Expressing this in Java is a two-step process:

- Step 1: Define a reference type for our hypernym (List61B.java).
- Step 2: Specify that SLLists and ALists are hyponyms of that type.



Step 1: Defining a List61B

We'll use the new keyword **interface** instead of **class** to define a List61B.

- Idea: Interface is a specification of what a List is able to do, not how to do it.

Step 1: Defining a List61B

We'll use the new keyword **interface** instead of **class** to define a List61B.

- Idea: Interface is a specification of what a List is able to do, not how to do it.

```
public interface List61B<Item> {  
    public void addFirst(Item x);  
    public void addLast(Item y);  
    public Item getFirst();  
    public Item getLast();  
    public Item removeLast();  
    public Item get(int i);  
    public void insert(Item x, int position);  
    public int size();  
}
```

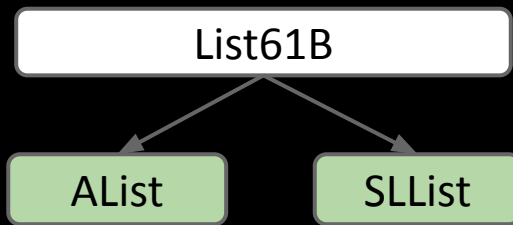
List61B

Step 2: Implementing the List61B Interface

We'll now:

- Use the new **implements** keyword to tell the Java compiler that SLList and AList are hyponyms of List61B.

```
public class AList<Item> implements List61B<Item>{  
    ...  
    public void addLast(Item x) {  
        ...  
    }  
}
```



We can now adjust our longest method to work on either kind of list:



```
public static String longest(List61B<String> list) {  
    int maxDex = 0;  
    for (int i = 0; i < list.size(); i += 1) {  
        String longestString = list.get(maxDex);  
        String thisString = list.get(i);  
        if (thisString.length() > longestString.length()) {  
            maxDex = i;  
        }  
    }  
  
    return list.get(maxDex);  
}
```

```
AList<String> a = new AList<>();  
a.addLast("egg");  
a.addLast("boyz");  
longest(a);
```

Overriding vs. Overloading

Lecture 9, CS61B, Spring 2026

Interface Inheritance

- The Desire for Generality
- Hypernyms and Hyponyms
- Interface and Implements
- Keywords
- **Overriding vs. Overloading**
- Interface Inheritance

Implementation Inheritance

- Default Methods
- Overriding Default Methods
- vs. Interface Inheritance

Collection Abstract Data Types

Changes to Scope in 61B

Method Overriding

If a “subclass” has a method with the exact same signature as in the “superclass”, we say the subclass **overrides** the method.

```
public interface List61B<Item> {  
    public void addLast(Item y);  
    ...  
}
```

```
public class AList<Item> implements List61B<Item>{  
    ...  
    public void addLast(Item x) {  
        ...  
    }  
}
```

AList **overrides** addLast(Item)

Method Overriding vs. Overloading

If a “subclass” has a method with the exact same signature as in the “superclass”, we say the subclass **overrides** the method.

- Animal’s subclass Pig overrides the makeNoise() method.
- Methods with the same name but different signatures are **overloaded**.
- The signature of a method is its name and the number and type of params.

```
public interface Animal {  
    public void makeNoise();  
}
```

```
public class Pig implements Animal {  
    public void makeNoise() {  
        System.out.print("oink");  
    }  
}
```

Pig **overrides** makeNoise()

```
public class Dog implements Animal {  
    public void makeNoise(Dog x) {  
        ...  
    }  
}
```

makeNoise is **overloaded**

```
public class Math {  
    public int abs(int a)  
    public double abs(double a)
```

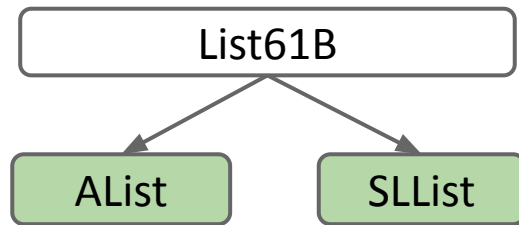
abs is **overloaded**

Optional Step 2B: Adding the @Override Annotation

In 61b, we'll always mark every overriding method with the **@Override** annotation.

- Example: Mark AList.java's overriding methods with **@Override**.
- The only effect of this tag is that the code won't compile if it is not actually an overriding method.

```
public class AList<Item> implements List61B<Item>{  
    ...  
  
    @Override  
    public void addLast(Item x) {  
        ...  
    }  
}
```



If a subclass has a method with the exact same signature as in the superclass, we say the subclass **overrides** the method.

- Even if you don't write `@Override`, subclass still overrides the method.
- `@Override` is just an optional reminder that you're overriding.

Why use `@Override`?

- Main reason: Protects against typos.
 - If you say `@Override`, but it the method isn't actually overriding anything, you'll get a compile error.
 - e.g. `public void addLats(Item x)`
- Reminds programmer that method definition came from somewhere higher up in the inheritance hierarchy.

Interface Inheritance

Lecture 9, CS61B, Spring 2026

Interface Inheritance

- The Desire for Generality
- Hypernyms and Hyponyms
- Interface and Implements
- Keywords
- Overriding vs. Overloading
- **Interface Inheritance**

Implementation Inheritance

- Default Methods
- Overriding Default Methods
- vs. Interface Inheritance

Collection Abstract Data Types

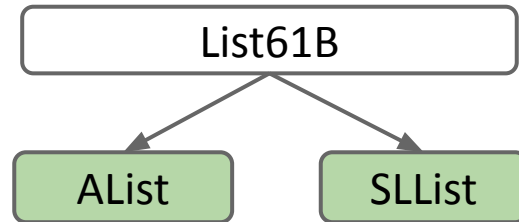
Changes to Scope in 61B

Interface Inheritance

Specifying the capabilities of a subclass using the **implements** keyword is known as **interface inheritance**.

- Interface: The list of all method signatures.
- Inheritance: The subclass “inherits” the interface from a superclass.
- Specifies what the subclass can do, but not how.
- Subclasses must override all of these methods!
 - Will fail to compile otherwise.

```
public interface List61B<Item> {  
    public void addFirst(Item x);  
    ...  
    public void proo();  
}
```

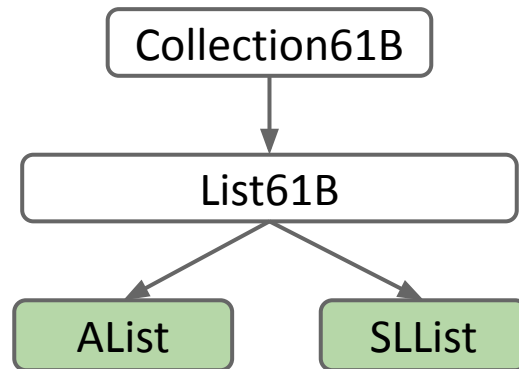


If AList doesn't have a proo() method,
AList will not compile!

Interface Inheritance

Specifying the capabilities of a subclass using the **implements** keyword is known as **interface inheritance**.

- Interface: The list of all method signatures.
- Inheritance: The subclass “inherits” the interface.
- Specifies what the subclass can do, but not how.
- Subclasses must override all of these methods!
- Such relationships can be multi-generational.
 - Figure: Interfaces in white, classes in green.
 - We'll talk about this in a later lecture.



Interface inheritance is a powerful tool for generalizing code.

- `WordUtils.longest` works on SLists, ALists, and even lists that have not yet been invented!

Recall: A memory box can only hold 64 bit addresses for the appropriate type.

- Example: `inputList` can only hold a `List61B<String>`.
- An `AList` is-a `List61B`, so `inputList` can hold a reference to the `AList`.

```
public static String longest(List61B<String> inputList) {  
    int maxDex = 0;  
    for (int i = 0; i < inputList.size(); i += 1)  
        ...  
}
```

```
public static void main(String[] args) {  
    AList<String> a1 = new AList<String>();  
    a1.addLast("horse");  
    WordUtils.longest(a1);  
}
```

Allowed! An
`AList` is a
`List61B`.



Will the code below compile? If so, what happens when it runs?

- a. Will not compile.
- b. Will compile, but will cause an error at runtime on the **new** line.
- c. When it runs, an **SLList** is created and its address is stored in the **someList** variable, but it crashes on **someList.addFirst()** since the **List** interface doesn't implement **addFirst**.
- d. When it runs, an **SLList** is created and its address is stored in the **someList** variable. Then the string "elk" is inserted into the **SLList** referred to by **addFirst**.

```
public static void main(String[] args) {  
    List61B<String> someList = new SLList<>();  
    someList.addFirst("elk");  
}
```

Question

Will the code below compile? If so, what happens when it runs?

- a. Will not compile.
- b. Will compile, but will cause an error at runtime on the **new** line.
- c. When it runs, an **SLList** is created and its address is stored in the **someList** variable, but it crashes on **someList.addFirst()** since the **List** interface doesn't implement **addFirst**.
- d. **When it runs, an SLList is created and its address is stored in the someList variable. Then the string "elk" is inserted into the SLList referred to by addFirst.**

```
public static void main(String[] args) {  
    List61B<String> someList = new SLList<>();  
    someList.addFirst("elk");  
}
```

Quick Reflection on LLMs

```
public Particle(ParticleFlavor flavor) {  
    this.flavor = flavor;  
    lifespan = flavor == ParticleFlavor.PLANT || flavor == ParticleFlavor.FLOWER || flavor == ParticleFlavor.FIRE? LIFESPANS.get(flavor): -1;  
}
```

```
return flavor == ParticleFlavor.SAND? Color.YELLOW: flavor == ParticleFlavor.BARRIER? Color.GRAY:  
    flavor == ParticleFlavor.WATER? Color.BLUE: flavor == ParticleFlavor.FOUNTAIN? Color.CYAN: Color.BLACK;
```

Default Methods

Lecture 9, CS61B, Spring 2026

Interface Inheritance

- The Desire for Generality
- Hypernyms and Hyponyms
- Interface and Implements
- Keywords
- Overriding vs. Overloading
- Interface Inheritance

Implementation Inheritance

- **Default Methods**
- Overriding Default Methods
- vs. Interface Inheritance

Collection Abstract Data Types

Changes to Scope in 61B

Interface inheritance:

- Subclass inherits signatures, but NOT implementation.

For better or worse, Java also allows **implementation inheritance**.

- Subclasses can inherit signatures AND implementation.

Use the **default** keyword to specify a method that subclasses should inherit from an **interface**.

- Example: Let's add a default print() method to List61B.java

Default Method Example: print()

```
public interface List61B<Item> {  
    public void addFirst(Item x);  
    public void addLast(Item y);  
    public Item getFirst();  
    public Item getLast();  
    public Item removeLast();  
    public Item get(int i);  
    public void insert(Item x, int position);  
    public int size();  
    default public void print() {  
        for (int i = 0; i < size(); i += 1) {  
            System.out.print(get(i) + " ");  
        }  
        System.out.println();  
    }  
}
```




Is the print() method efficient?

- a. Inefficient for AList and SLList
- b. Efficient for AList, inefficient for SLList
- c. Inefficient for AList, efficient for SLList
- d. Efficient for both AList and SLList

```
public interface List61B<Item> {  
    ...  
    default public void print() {  
        for (int i = 0; i < size(); i += 1) {  
            System.out.print(get(i) + " ");  
        }  
        System.out.println();  
    }  
}
```

Overriding Default Methods

Lecture 9, CS61B, Spring 2026

Interface Inheritance

- The Desire for Generality
- Hypernyms and Hyponyms
- Interface and Implements Keywords
- Overriding vs. Overloading
- Interface Inheritance

Implementation Inheritance

- Default Methods
- **Overriding Default Methods**
- vs. Interface Inheritance

Collection Abstract Data Types

Changes to Scope in 61B

Question

Is the print() method efficient?

- a. Inefficient for AList and SLList
- b. Efficient for AList, inefficient for SLList**
- c. Inefficient for AList, efficient for SLList
- d. Efficient for both AList and SLList

```
public interface List61B<Item> {  
    ...  
    default public void print() {  
        for (int i = 0; i < size(); i += 1) {  
            System.out.print(get(i) + " ");  
        }  
        System.out.println();  
    }  
}
```

get has to seek all the way to the given item for SLists.

Overriding Default Methods

If you don't like a default method, you can override it.

- Any call to `print()` on an `SLList` will use this method instead of default.
- Use (optional) `@Override` to catch typos like `public void pirnt()`

```
public class SLList<Item> implements List61B<Item> {  
    @Override  
    public void print() {  
        for (Node p = sentinel.next; p != null; p = p.next) {  
            System.out.print(p.item + " ");  
        }  
  
        System.out.println();  
    }  
}
```

Recall that if X is a superclass of Y, then an X variable can hold a reference to a Y.

Which print method do you think will run when the code below executes?

- List.print()
- SLList.print()

```
public static void main(String[] args) {  
    List61B<String> someList = new SLList<>();  
    someList.addLast("elk");  
    someList.addLast("are");  
    someList.addLast("watching");  
    someList.print();  
}
```

Recall that if X is a superclass of Y, then an X variable can hold a reference to a Y.

Which print method do you think will run when the code below executes?

- List.print()
- **SLList.print() : Yes**
 - Since the object we're pointing at has overridden print, we'll use that overridden method.

```
public static void main(String[] args) {  
    List61B<String> someList = new SLList<>();  
    someList.addLast("elk");  
    someList.addLast("are");  
    someList.addLast("watching");  
    someList.print();  
}
```

vs. Interface Inheritance

Lecture 9, CS61B, Spring 2026

Interface Inheritance

- The Desire for Generality
- Hypernyms and Hyponyms
- Interface and Implements Keywords
- Overriding vs. Overloading
- Interface Inheritance

Implementation Inheritance

- Default Methods
- Overriding Default Methods
- **vs. Interface Inheritance**

Collection Abstract Data Types

Changes to Scope in 61B

Interface Inheritance (a.k.a. what):

- Allows you to generalize code in a powerful, simple way.

Implementation Inheritance (a.k.a. how):

- Allows code-reuse: Subclasses inherit code from interfaces.
 - Example: `print()` implemented in `List61B.java`.
 - Gives another dimension of control to subclass designers: Can decide whether or not to override default implementations.
- We'll see later (briefly) how implementation inheritance can be abused.

Collection Abstract Data Types

Lecture 16, CS61B, Spring 2023

Interface Inheritance

- The Desire for Generality
- Hypernyms and Hyponyms
- Interface and Implements Keywords
- Overriding vs. Overloading
- Interface Inheritance

Implementation Inheritance

- Default Methods
- Overriding Default Methods
- vs. Interface Inheritance

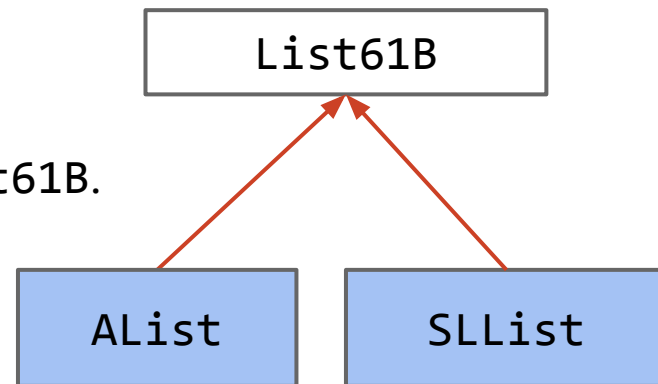
Collection Abstract Data Types

Changes to Scope in 61B

Interfaces vs. Implementation

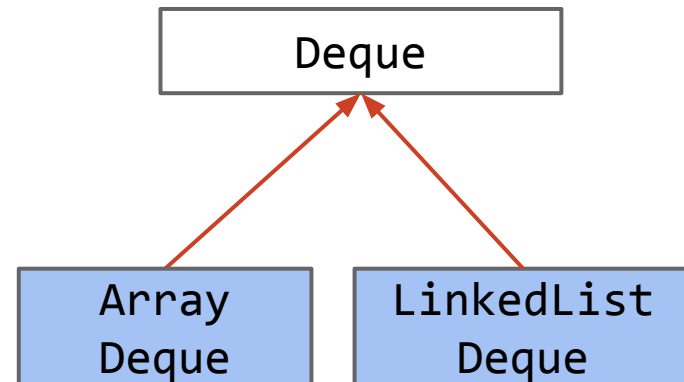
In class:

- Developed ALists and SLLists.
- Created an interface List61B.
 - Modified AList and SLList to implement List61B.
 - List61B provided default methods.



In projects:

- Will develop ArrayDeque and LinkedListDeque.
 - Each class implements the Deque interface.

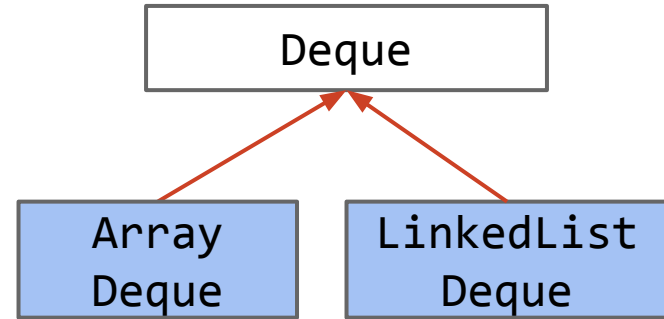


Abstract Data Types

An **Abstract Data Type (ADT)** is defined only by its operations, not by its implementation.

Deque ADT:

- `addFirst(Item x);`
- `addLast(Item x);`
- `boolean isEmpty();`
- `int size();`
- `printDeque();`
- `Item removeFirst();`
- `Item removeLast();`
- `Item get(int index);`



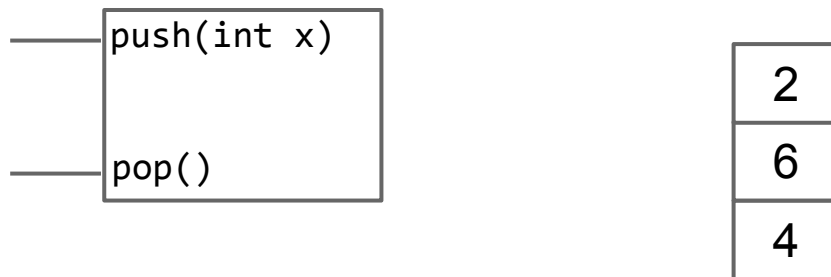
ArrayDeque and LinkedList Deque are implementations of the Deque ADT.



Another example of an ADT: The Stack

The Stack ADT supports the following operations:

- `push(int x)`: Puts `x` on top of the stack.
- `int pop()`: Removes and returns the top item from the stack.



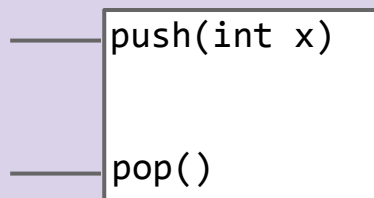


The Stack ADT supports the following operations:

- `push(int x)`: Puts `x` on top of the stack.
- `int pop()`: Removes and returns the top item from the stack.

Which implementation do you think would result in faster overall performance?

- A. Linked List
- B. Array



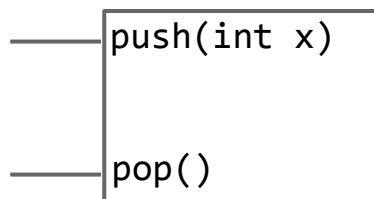
The Stack ADT supports the following operations:

- `push(int x)`: Puts `x` on top of the stack.
- `int pop()`: Removes and returns the top item from the stack

Which implementation do you think would result in faster overall performance?

A. Linked List

B. Array



4

Both are about the same. No resizing for linked lists, so probably a lil faster.



The GrabBag ADT supports the following operations:

- `insert(int x)`: Inserts `x` into the grab bag.
- `int remove()`: Removes a random item from the bag.
- `int sample()`: Samples a random item from the bag (without removing!)
- `int size()`: Number of items in the bag.

Which implementation do you think would result in faster overall performance?

- A. Linked List
- B. Array

```
— insert(int x)
— remove()
— sample()
— size(int i)
```

The GrabBag ADT supports the following operations:

- `insert(int x)`: Inserts `x` into the grab bag.
- `int remove()`: Removes a random item from the bag.
- `int sample()`: Samples a random item from the bag (without removing!)
- `int size()`: Number of items in the bag.

Which implementation do you think would result in faster overall performance?

A. Linked List

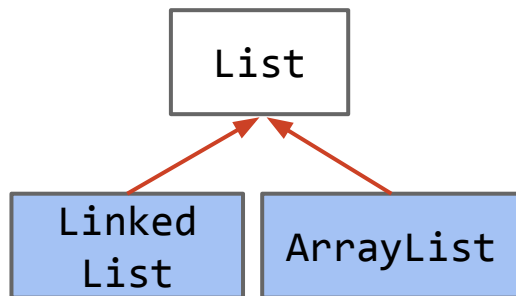
B. Array

```
— insert(int x)
— remove()
— sample()
— size(int i)
```


One thing I particularly like about Java is the syntax differentiation between abstract data types and implementations.

- Note: Interfaces in Java aren't purely abstract as they can contain some implementation details, e.g. default methods.

Example: `List<Integer> L = new ArrayList<>();`

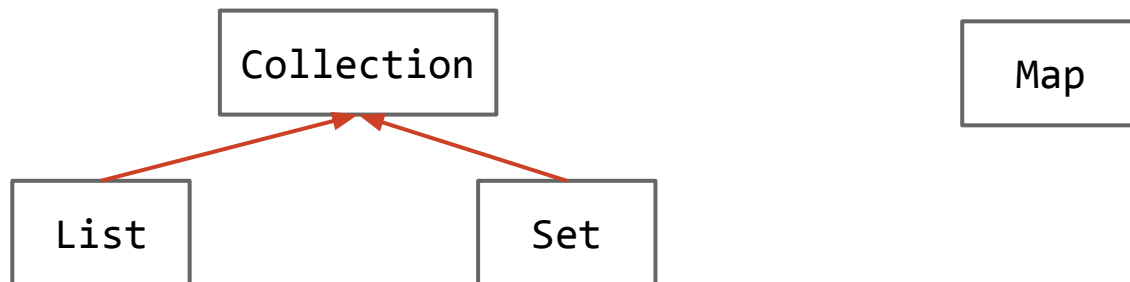


We've already seen several examples interfaces in Java:

- Lists of things.
- Sets of things.
- It turns out that Lists and Sets are subinterfaces of a more abstract interface called a Collection. Won't really discuss in 61B.

And of course we've seen the Map, e.g. jhug's grade is 88.4.

- Maps are also known as associative arrays, associative lists (in Lisp), symbol tables, or dictionaries (in Python).



Changes to Scope in 61B

Lecture 9, CS61B, Spring 2026

Interface Inheritance

- The Desire for Generality
- Hypernyms and Hyponyms
- Interface and Implements Keywords
- Overriding vs. Overloading
- Interface Inheritance

Implementation Inheritance

- Default Methods
- Overriding Default Methods
- vs. Interface Inheritance

Collection Abstract Data Types

Changes to Scope in 61B

61B used to teach something called “dynamic method selection.” It relied on the concepts of the “static type” (a.k.a. “compile-time type”) and the “dynamic type” (a.k.a. “run-time type”).

Students spent a great deal of time on something that isn’t ultimately very important. This is not a class about Java minutiae, so I cut this material.

- Example, the infamous Bird/Falcon/gulgate problem from Spring 2017:
https://hkn.eecs.berkeley.edu/examfiles/cs61b_sp17_mt1.pdf
- If you really want to go there, see sp21 61b slides:
https://docs.google.com/presentation/d/1EUOpd9NXq28eEUqQMZoo_rRpxw8EekQ2LEp61U9bnDc/edit#slide=id.g30617fc64e_34_32

Note: We will cover the **extends** keyword after the midterm. It is not in scope for midterm 1.